

04-实施指南-P1-训练数据流加速系统

P1 实施指南：训练数据流加速系统（代号 TrainCache）——v3 重构版

2026-10-05 v3 重构（导师与学员讨论定稿）：修正 v1 的指标错位、对照组缺失、术语硬伤与过时负载模型。结构总览见 03-项目结构定稿.md（v3），本文为 P1 详细实施指南。配套：07-开源贡献指南.md（本项目上游回馈目标改为 **Fluid**）。目标仍是“照着能开工”。

1. 项目介绍

电梯陈述：面向 K8s 训练 Job 的”预热-调度闭环”系统——把工业方案中松耦合的两件事（手动触发的数据预热、粗粒度的缓存节点亲和）做成由调度决策驱动的闭环：按 Job 实际消费的数据子集、以块级粒度向候选节点预热，以细粒度就绪度门控/打分作业，超时可降级；并用**端到端 makespan** 与真实部署的 Fluid/JuiceFS 上秤，量化它在什么条件下赢、在什么条件下反而更慢。

三重定位（与 P2 同构，缺一不可）：

- 对标者：**Fluid（Dataset/DataLoad/Runtime + 缓存亲和调度）与 JuiceFS CE 作为一等对照组真实部署、同负载跑分，不打空气靶；
- 测量者：**头号指标是 Job 提交→首个 step 与整个 epoch 的 makespan，预热的重复字节成本计入分母；允许并预注册阴性结论；
- 改进者：**现勘证实的缺口以测量数据支撑，向 **Fluid（Apache-2.0、Go、调度/预热正是其代码域）** 提 issue/PR，而不是停在”我自己的仓库里有这个功能”。

明确不是什么：不是又一个分布式文件系统（不重造 JuiceFS 的 POSIX/FUSE 数据面）；不是缓存算法论文（W-TinyLFU 是工程对照不是学术声称）；不是需要 GPU 的项目——全项目在 CPU 训练循环上即可测，测的是 dataloader stall，不是 GPU 利用率。

2. 问题与动机（含 2026-10 生态现勘）

2.1 外部事实（2026-10-05 核实，构成立项边界）

- Fluid 的缓存亲和调度是”粗粒度二进制 + 事后注入”：**mutating webhook 在 Pod 创建时读取其引用的 Dataset，向 Pod spec 注入 `nodeAffinity`（ACK 增强文档给出的默认权重：缓存节点 100 / 同可用区 50 / 同地域 20，打标签可升级为硬亲和），另有 `RequireNodeWithFuse`（要求节点已有 FUSE 客户端）与 `PreferNodesWithoutCache`（把不消费数据集的 Pod 引开）；若 Pod 自带 affinity/nodeSelector 则跳过注入。它回答的问题是”节点上**有没有**这个 Dataset 的 runtime/缓存”，**不知道缓存了多少比例、哪些块**。

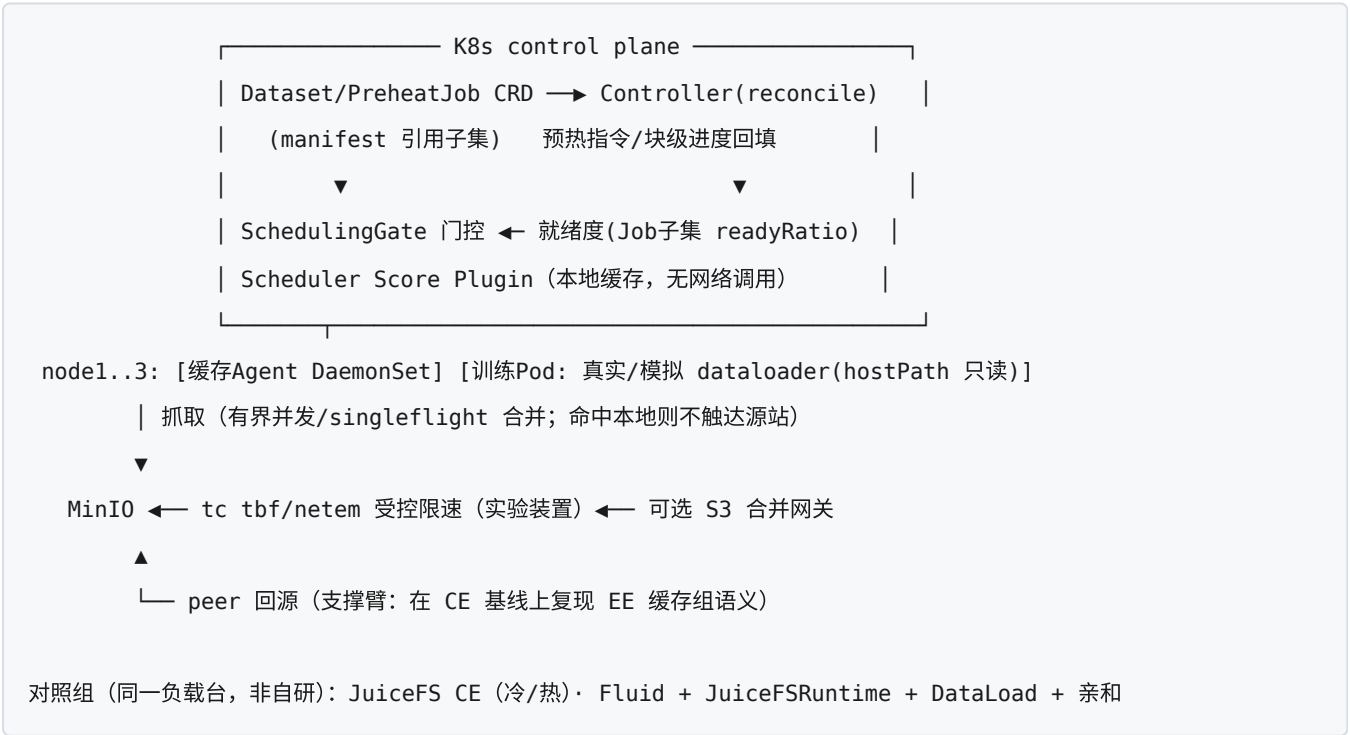
- **预热 (DataLoad) 是独立、手动创建的 CRD**，与调度决策不闭环：DataLoad 不按”某个 Job 即将被调度”驱动，调度也不按 DataLoad 的块级进度门控。开源 Fluid 与 ACK 托管增强 (ack-fluid, ConfigMap 可配权重的那套) 的能力边界必须在现勘阶段划清。
- **JuiceFS 社区版 v1.3.1 为 Apache-2.0 许可** (早期”AGPLv3”的说法已过时)：4MiB block、warmup、`--prefetch` 预读、单机本地缓存；**分布式缓存组 (一致性哈希环、peer 间共享块)、IORPC、RDMA 均为企业版能力** (EE 5.2/5.3+)。本项目 4MiB 块大小与 CE 默认值对齐，作为报告注脚。
- **2026 年训练数据加载主流形态**：webdataset tar 分片 / Mosaic MDS 这类百 MB 级分片 + 确定性 shuffle 的流式读取；百万小文件 ImageNet 模式是元数据压力场景，不再是 LLM/大模型训练的典型 I/O 形态。

2.2 仍然成立的问题 (本项目的缝)

1. **预热与调度不闭环**：工业方案里”先手动 DataLoad、再靠二进制亲和碰运气”之间，没有”Job 即将调度 → 向哪几个候选节点预热它需要的那些块 → 块级就绪度决定何时放行/如何打分 → 超时如何降级”的控制回路；
2. **就绪度粒度错配**：节点”有缓存 runtime”与”这个 Job 要读的分片 90% 在本地”是两回事；多 rank/分片训练下，Job 往往只消费数据集的一个子集；
3. **预热成本无人记账**：向 2-3 个候选节点重复预热有 2-3 倍字节与带宽代价，什么情况下这笔代价值得，没有公开曲线。

立项闸门 (与 P2 同规矩)：§ 2.1 的缺口判断必须在阶段 0 由**开源 Fluid 实测取证**确认 (webhook 注入内容、DataLoad 与调度的时序关系)；若现勘发现开源 Fluid 已提供块级进度门控，叙事收缩为”复现与量化”，不硬造缺口。

3. 架构总览



数据面约定（保留 v1 决策）：preheat 模型——Agent 把块落到 `/var/cache/traincache/<ds>/blocks/`，训练 Pod 用 hostPath PV 只读挂载，直接走内核 page cache，**Agent 不参与读 I/O 路径**：读路径零额外延迟。代价是读侧信号靠旁路（§ 5.4）。FUSE read-through 降为可选 stretch（Fluid 生态已有 FUSE/CSI，不重造）。

4. 阶段 0：现勘（与使用阶段合并，1–2 周，闸门制）

4.0 可行性探针（最先做，2–3 天，写任何控制器代码之前）

目的：用几十行脚本先证明实验方法论成立——装置造不出可信差异，三个月后改设计的成本是整个项目。这是全 P1 性价比最高的三天。

- **探针 A：冷热差稳定可复现。**同一批百 MB 分片，用 `tc qdisc ... tbf rate 100mbit burst ... netem delay 20ms` 挂在 Agent→MinIO 路径；冷读（`sync; echo 3 > drop_caches` + 空本地缓存，走 tc）与热读（本地缓存命中，不经 tc）各 ≥ 7 次。
 - **绿灯标准：**冷/热中位延迟差 $\geq 5\times$ （或中位吞吐差一个数量级）；两组 bootstrap 95% CI **不重叠**；组内变异系数 $CV \leq 15\%$ ；
 - **红灯不是去硬调参数凑数字，而是排查装置：**tc 是否挂在正确网卡/出口（`tc -s qdisc` 看计数是否在涨）、热读是否真的零网络（`ss /tcpdump` 验证未建连）、单盘自身是否已成瓶颈、MinIO 自己的 page cache 是否制造了“假冷读”；
- **探针 B：stall 指标打得出。**最小 CPU PyTorch DataLoader（webdataset 读真 JPEG tar，`num_workers=0/4/8` 三档，无模型或极小矩阵乘），训练循环里打点 `wait_time/step_time`。
 - **绿灯标准：**冷态 stall 占比稳定 $>40\%$ 、热态 $<15\%$ ；增加 workers 冷态 stall 单调下降（证明测到的是真实 I/O 饥饿而非打点噪声）；
- **硬时间盒 3 天。**第 3 天晚仍红灯 → 停止开工，复盘根因（必要时上外接 SSD 或提前租两盘 CPU 云主机），**不带病进入步骤 1。**

动作清单：

1. kind ≥ 1.26 集群（PodSchedulingGate 前置）部署 MinIO、**JuiceFS CE（本地缓存模式）、开源 Fluid + JuiceFSRuntime**；
2. 跑通官方 demo：Dataset → DataLoad 手动预热 → 带 dataset 标签的训练 Pod；抓取并归档 **webhook 实际注入的 Pod spec**（nodeAffinity 权重、FUSE 注入）、DataLoad 与 Pod 调度的**时序**（Pod 是否等 DataLoad 完成？还是先调度后等数据？）；
3. 划清能力边界表：开源 Fluid vs ACK 文档增强能力 vs JuiceFS CE vs EE（缓存组/RDMA）；每个判断附版本号与证据（kubectl 输出/文档链接/截图）；
4. 用 JuiceFS `warmup` 完成第一次“对手热读”基线，记录版本与挂载参数；
5. 产出 **《P1 对手现勘报告》**（与 P2 现勘合并为同一时期的总交付物）。

闸门产出：①缺口被证据确认 → 核心 1 按“闭环”叙事推进；②缺口不成立 → 核心 1 改为“Fluid 机制的独立复现与参数量化”，不得带假设开工。

5. 基座设计

5.1 CRD 设计

```
apiVersion: traincache.io/v1alpha1
kind: Dataset
spec:
  source: { endpoint: http://minio:9000, bucket: datasets, prefix: shards/ }
  chunking: { strategy: fixed, chunkSize: 4MiB } # 与 JuiceFS 默认 block 对齐
  index: { algo: path-index } # 索引层: 路径寻址 (见 5.2)
  storage: { digest: xxh3-128, dedup: true } # 存储层: CAS
  version: v5 # 版本前缀目录, 不做自研失效协议 (见 §9.3)
status:
  totalChunks: 262144
  nodes: # 粗粒度状态, 细清单不下发 etcd
    - { node: node1, readyRatio: 0.98, bytes: 1.0Ti, inflightBytes: 64Gi }
  conditions: [{ type: Ready, status: "False", reason: Preheating }]
```

- **etcd 对象 1.5MB 上限**: 块级清单放对象存储上的 index shard (每 shard ≤ 10 万条), CRD status 只存每节点 readyRatio + shard 位图摘要 (roaring bitmap 序列化后的摘要哈希); 调度插件需要精确清单时按需拉 shard;
- **Job 子集引用**: 训练 Job (尤其多 rank) 通过 label/annotation 声明消费的 manifest 子集 (分片列表); 就绪度一律按**子集**计算, 不按全数据集;
- **version** 即源前缀的代次 (shards/v5/ 式前缀目录); index 构建做增量: ETag 未变的 shard 跳过, 避免百万对象全量 List。

5.2 双层标识模型 (修正 v1 术语硬伤)

v1 的 块ID = xxh3(path+index) 是**路径寻址加校验**, 不是内容寻址——v3 明确拆成两层:

- **索引层 (决定取什么)**: manifest 条目键 = (path, chunkIndex), 路径寻址。它不承诺跨路径去重;
- **存储层 (决定怎么落盘)**: 块在预热下载时**本来就要流过内存**, 流式计算 xxh3-128 (约 10GB/s/核, 零额外 I/O) 作为存储键, 落盘 blocks/ab/<contentHash>.blk + refcount 索引。于是:
 - 读回校验是内置能力 (块头元信息 64B 内记录大小/摘要, 不搞 sidecar);
 - **安全去重自动成立**: 不同路径的相同内容、多分片公共头部只占一份空间, refcount=0 才进入可驱逐集合;
- 措辞统一为”**路径索引 + 内容摘要存储 (CAS 存储层, 可选去重)**”。面试讲法: “寻址回答的是要什么, 摘要是落盘后算出来的, 不能反推路径——我在 v1 里把这两层混为一谈, 后来纠正了。”
- 哈希选型: 内部信任边界用 xxh3-128 换性能, 全量 SHA-256 校验留作开关; 碰撞概率与信任边界的权衡要能讲。

5.3 缓存 Agent (Go, 保留 v1 工程设计)

并发模型：

抓取管线 = workqueue(块, 优先级) → errgroup + 按字节加权信号量 → 分块写盘

限制 bytes-in-flight (默认 512MiB) 而非 goroutine 数：防小文件 goroutine 爆炸、大文件 OOM。

- HTTP 层 MaxConnsPerHost (默认 64) 与 MinIO 连接池对齐；优先级：显式预热 > 邻域预取 > 后台补全；
- **原子可见性**：先写 tmp 再 rename()；缓存块不逐块 fsync (可重拉数据，掉电丢块=重下) ——与存储系统直觉相反但正确的取舍；
- **驱逐**：双水位 (85% 触发到 75%)，LRU 序 + refcount 钉住在读块；unlink 已打开文件安全 (fd 可读)，驱逐不等读者；按 Dataset 分组配额；
- **启动重扫描**：重启后扫描 blocks 目录重建 CAS 索引，块头抽样校验 1%，全量校验后台低优先级——这是混沌韧性 (§ 8.4) 的地基。

5.4 读侧信号与命中率口径 (保留, 诚实标注)

- inotify 监听缓存目录 IN_OPEN (去抖：同 pid 同文件多次 open 合并，采样率可配)，得到打开频率与顺序；
- **命中率口径写进报告**：消费者侧——instrumented dataloader 上报打开块集合，与本地块集合求交；**不声称 Agent 读路径精确记账** (preheat 模型做不到)；进阶替代为 eBPF (S3 项目)。

6. 核心 1：预热-调度闭环 (项目灵魂)

把 v1 的增量 1 (gate) 与增量 2 (预热状态机) 合并为一个控制回路。

6.1 控制回路

Job 创建 (声明 Dataset + 消费子集 + 门控模式注解)

- Controller 注入 SchedulingGate: traincache.io/preheat-pending
- 按放置策略选候选节点 (fanout=1 / 2-3 / 全部, 本身是实验变量)
- 向候选 Agent 下发"子集块清单"预热任务 (字节预算 + 速率上限)
- Agent 块级进度回填 status
- 子集 readyRatio ≥ 阈值 (默认 0.9) → 摘除 gate
 - 或总超时 (默认 5m, 注解可覆盖) → 降级：去 gate/去亲和正常调度
- Score plugin 按"节点持有该 Job 子集块比例"打分 → bind

实现注意点 (保留 v1 教训)：

- gate 注入/摘除幂等 + finalizer 防孤儿；**k8s ≥ 1.26**, kind 镜像版本在阶段 0 确认；
- **Score plugin 内不做网络调用**：informer watch 聚合 status, 本地 cache + 1s TTL——把 scheduler 打挂是调度器扩展的经典坑；

- 打分：`score(node) = readyRatio_subset` 为主，次序空闲磁盘/在途预热负载；门控模式分两种：`required`（硬等，不到阈值只走超时降级）与 `preferred`（软门控，超时即放行）；
- 降级语义：不是报错而是”去掉约束继续调度”，`Condition=Degraded + Event + Prometheus` 指标；数据源不可用时总超时兜底，**Job 永不因预热死锁 Pending**；
- 预热速率受预算约束（默认不超过节点可观测空闲带宽的一定比例），防止预热点火挤压在线 Job。

6.2 预注册假设与盈亏平衡旗舰图

v1 的致命问题是只度量 gate 摘除后的等待时间，不记账预热成本。v3 头号指标改为：

类别	指标
效果	Job 提交 → 首个 step 时间；整个 epoch makespan； dataloader stall 占比（等待/总时长，CPU 训练循环直接打点，无需 GPU）
成本	预热总字节 × fanout（重复预热倍数）；源站出口字节； 预热带宽对在线流量的挤压（在跑 Job 的 stall 扰动）
过程	各候选节点 readyRatio 时间序列；降级触发率；门控等待时长

实验开始前写死假设（预注册，数字长什么样报什么样）：

- 门控策略**预期赢**：①多 epoch/多轮重复训练 ②数据集 > 缓存容量但热子集稳定 ③多 Job 共享数据 ④源站慢/带宽贵（tc 高延迟限速档）；
- 门控策略**预期可能输**：①单次全量顺序 epoch（预热和训练读的是同一遍数据，门控只增加串行化）②缓存容量 >> 数据集 ③源站极快 ④预热超时频繁（预热不如不预）；
- **旗舰图 = 盈亏平衡曲面**：源站带宽 × 数据集/缓存容量比 × fanout 三维扫点（或固定一维画二维曲线），画出”闭环门控 makespan 减自由调度 makespan”的正负区间。这张图即使大半区域为负也是成功——它回答”这东西什么时候不该用”，而这正是面试官最想听的工程判断。
- 配套次旗舰图：fanout=1/2/3 的 **makespan 收益 vs 重复字节成本** Pareto 散点。

7. 核心 2：欠配缓存下的准入与预取（场景修正，接受阴性）

7.1 为什么 v1 的 W-TinyLFU 旗舰图可能是平线

全量顺序 epoch + 缓存容量够大时，LRU/LFU/任何准入策略命中率趋同——准入差异只在**缓存欠配 + 访问倾斜 + 热子集被重复消费 + 多租户互相挤占**时才出现。实验必须主动构造这个场景，而不是在顺序扫描上比三个算法。

7.2 实验设计

- **多 Job 倾斜 trace（负载 C 轨，§ 10.1）**：Job A 反复读 10% 热分片（模拟多轮微调/重复 epoch），Job B 顺序扫全量（流寇式扫描），缓存容量设为数据集的 30%；
- 对照：LRU / LFU / W-TinyLFU 三策略下，**Job A 的命中率与 stall 占比**、扫描作业对热集的驱逐次数；

- W-TinyLFU 实现（保留 v1 规格）：window（SLRU probationary/protected）+ count-min sketch（4 行 × 10 × 缓存块数、3bit 饱和计数）+ doorkeeper bloom；准入规则 $\text{freq}(\text{candidate}) > \text{freq}(\text{victim})$ 才换入；定期 halving；参数（window 占比、halving 周期）扫点出敏感性图；
- **自适应预取**：inotify 观测块 i 被打开 → 抓后继 K=8 块；预取命中率低于阈值则降 K；
- **允许的结论**：若全量顺序场景三策略无差异、倾斜场景有差异——“照实写”准入策略对单 Job 顺序训练不关键，关键在预取调度与跨租户保护”。这是成熟的阴性发现，不是实验失败。

8. 支撑臂设计

8.1 S3 合并回源网关（保留 v1）

- singleflight 按对象 key 合并并发 GET；大对象用 io.Pipe /splice 流式扇出，禁止整块缓冲分发；
- AIMD 背压：MinIO P99 上升/5xx 时令牌桶减半，平稳期线性恢复；并发上限即“对中心存储的礼貌”；
- sync.Pool 复用块级 buffer，以 GODEBUG=gctrace=1 对比 alloc/B 与 GC pause（S5 热身）。

8.2 P2P 块共享：在 CE 基线上复现 EE”缓存组”语义（定位重写）

v1 的增量 5 是普通 P2P；v3 明确其对标对象：**JuiceFS 企业版的分布式缓存组（一致性哈希环定位块、peer 回源、节点增删迁移）**，用 Go 自研组件在只有单机缓存的 CE 基线上把这套语义做出来：

- 发现复用 CRD status：节点上报块 shard 位图；一致性哈希决定块的”老家节点”；
- 拉取序：peer 持有 → 限 2-3 个 peer 多源并发 → 否则回源；peer 失败立即穿透回源（与 EE 文档描述的故障语义一致：peer 故障不致命）；
- **全局带宽预算（P2C 思想，保留）**：每节点令牌桶，peer 与 origin 流量共用，origin 保底 20%，防止 P2P 挤掉训练流量；
- 传输抽象 Transport：gRPC stream 为默认实现；**RDMA 作为实验臂插在接口后**（§ 9.1），永不进主路径；
- 对照指标：源站出口字节、多节点冷启动 makespan；性能数字与 EE 官方口径只做量级参照，明确标注条件不同不可直接比。

8.3 Checkpoint 异步 sidecar（修正 v1 语义错误）

- **写路径（保留）**：trainer 写共享 emptyDir → sidecar fsnotify → 16MB 定长分片 + 逐片 xxh3 → multipart 并发上传（8 片）→ **最后写 manifest 对象作为发布点**（S3 上永远只见完整 checkpoint）；multipart ETag 非 md5，校验以 manifest 逐片哈希为准；
- **恢复路径（修正）**：v1”权重先到、模型先起、optimizer state 后到”对**训练恢复是错误的**——optimizer state（如 Adam 动量）缺失时 stepping 会改变收敛轨迹。正确分两场景：
 - **推理/eval**：权重到位即可用，成立；
 - **训练恢复**：权重 + optimizer state **完整后才允许第一个 step**；sidecar 的真实价值是”异步分片上传不阻塞训练循环 + 恢复时 Range GET 并发拉取 + 加载与初始化重叠缩短 stall”，而不是省状态；
- 指标：checkpoint 写入对训练 step 的扰动、恢复总时长拆解（拉取/加载/初始化重叠）。

8.4 混沌套件（保留 v1 四件套）

注入	手段	期望
Agent kill -9	崩溃 + 重启	重扫描恢复索引，无数据错误
网络劣化	tc/netem（privileged 限测试容器）	背压生效，P99 恶化但有界
位翻转	测试钩子改块字节	校验失败 → 重拉 → 有指标
数据源宕机	scale MinIO 到 0	门控超时降级，Job 不卡死

9. 调研/实验臂（只做实验或只写文档，绝不进主路径）

9.1 RDMA 传输臂（无硬件的可行路径，承接学员个人兴趣）

- **笔记本上跑真 verbs 代码**：Soft-RoCE（内核 `rdma_rxe`）提供完整 `libibverbs/librdmacm` 语义（QP 状态机、MR 注册、RC/UD），API 与真卡一致；
- **铁律：Soft-RoCE 的性能数字一律不进报告**（绕内核 UDP 封装，常比 TCP 更慢），只验证功能正确性、连接管理、故障语义；kind 内 `rxe` 有网络命名空间坑，开发先在宿主机/ `--network host` 特权容器进行；
- **真值按小时租**：阿里云 eRDMA（ERI），普通 c7/g7/r7 规格即可挂载，两台同可用区小规格 CPU 实例对测一下午（量级几十元），数字标注一次性校准环境；
- 实验内容：4MiB 块 TCP vs RDMA 的吞吐/CPU/拷贝路径；MR 注册开销与 MR cache；**不通过 cgo 把 libibverbs 链进 Go 热进程**——C++ 独立传输组件 + 薄 C ABI（与 S2 能力线合流）；
- 位置：§ 8.2 的 `Transport` 实现 B，排在 gRPC 基线之后，永远是实验臂。

9.2 io_uring / S2（收益预期预先诚实化）

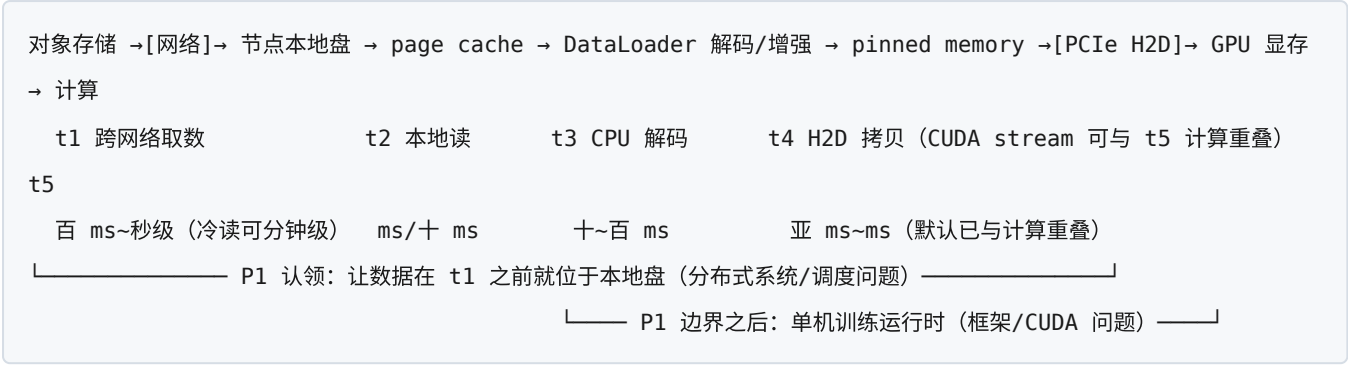
4MiB 大块经 HTTP 抓取是**网络 bound**，本地写路径省 `syscall` 对吞吐贡献预期≈0；`io_uring` 的战场在**小文件高 IOPS 臂（负载 A 轨）**。报告的正确姿势是” `syscall` 数大降、大块吞吐不变并解释原因、小文件臂 IOPS 提升 X”。`O_DIRECT` 三重对齐（偏移/长度/buffer 地址）、注册缓冲与 `fio` 校准照做。

9.3 版本化（v1 增量 7 砍掉自研协议）

版本前缀目录（`datasets/imagenet/v5/...`）天然消解失效问题，版本寻址即失效边界。写一页 ADR 说明”为什么不造失效协议”，测试只覆盖版本切换、`rename`、重提交路径。GPU 侧技术（DALI/GDS/CUDA Stream）的知识边界见 § 9.4。

9.4 GPU 侧技术知识边界（DALI / GDS / CUDA Stream：只调研、半页笔记、不动手）

一个训练 step 的数据路径与归属：



- **DALI**: GPU 侧图像解码/增强, 消灭 t3; **GDS (GPUDirect Storage)**: NVMe→显存 DMA 直传、绕过 CPU bounce buffer 与 page cache, 优化 t2→t4 的毫秒级拷贝, 需文件系统/硬件/驱动全链路支持, 不具备时自动回退兼容路径; **CUDA Stream**: 让 t4 与 t5 重叠 (DataLoader prefetch 下本就重叠良好);
- **为什么不动手**: ①岗位分工——t1 是分布式系统问题 (Go/调度/缓存, Fluid/JuiceFS 全部止步于”文件在本地 POSIX 路径就绪”), t2-t4 是训练框架/CUDA 问题 (Python/C++, 对标 DALI/NVIDIA), 越界一层则五组对照与上游叙事断裂、身份从平台作品变框架半成品; ②量级——t1 通常比 t4 大 2-3 个数量级; ③真值——t2-t4 的性能结论必须真 GPU, 游戏本测不了, 拿不出的数字不是弹药;
- **必须能白板答的两个边界题**:
 1. **“训练用了 DALI, P1 还有价值吗?”** ——DALI 只消灭 t3 解码等待, 消灭不了跨网络取 shard 的 t1, DALI 自身仍要先读到原始分片; P1 让分片本地就绪依然成立, 但”解码不再是瓶颈”会使收益区间收窄, 作为盈亏曲面的讨论变量诚实标注;
 2. **“GDS 绕过 page cache, 你的缓存命中模型怎么办?”** ——GDS 绕过的是内核 page cache, 不是本地盘; P1 的核心价值是”数据在本地且预热完成”, GDS 从本地 NVMe 直传反而更依赖本地就绪; 要懂的是它的部署约束 (文件系统/硬件/驱动支持与回退), 不是把它做进来;
- **交付物**: 阶段 0 同期半页笔记 (本路径图 + 两问答案); 面试口径固定为”没动手做, 但知道它在流水线上和我的系统怎么接力”。

10. Benchmark 方法学 (可复现纪律)

10.1 三轨负载

轨	形态	定位
A 元数据压力臂	100 万 × 4KB 小文件 (脚本生成, 参数化)	元数据风暴/小文件 IOPS, io_uring 的战场; 不再自称训练主流
B 真实分片主臂	webdataset tar 或 Mosaic MDS, 百 MB 分片 + 真实 PyTorch DataLoader + 确定性 shuffle; 脚本合成真 JPEG 编码分片 (见下)	2026 年训练真实形态, 所有旗舰数字以此臂为准, 前移到步骤 5
C 多 Job 倾斜臂	Job A 反复读 10% 热分片 + Job B 全量扫描, 缓存欠配 30%	核心 2 专用

模拟器只用于参数扫点；旗舰数字必须有真实 dataloader 复跑。CPU 训练循环（无模型或极小模型）以 dataloader stall 占比为代理指标。

B 轨数据合成纪律（零下载、真解码）：不用纯随机 tensor（没有解码成本，被问”不真实”无法回答），也不下载真实 ImageNet（带宽/许可/时间）；脚本批量生成随机像素图像 → **真实 JPEG 编码**（PIL/turbojpeg，质量与分辨率参数化）→ 按 webdataset 规范打百 MB tar 分片，固定随机种子、一键再生成。这样解码是真的、CPU 压力是真的、shuffle 是真的；报告注明”合成数据，I/O 与解码路径真实、语义内容随机”，并把”合成图解码成本分布可能不同于真实 ImageNet”列为外推威胁，可选一次小样本真实数据做 sanity check。

10.2 笔记本上造出真实冷热差（关键实验装置）

单机一块物理盘不天然存在”本地 NVMe vs 网络取数”的速度差。解法：**用 tc qdisc tbf/netem 对 Agent→MinIO 的源站路径做受控限速 + 延迟**（如 100Mbps、20ms 档，并扫点模拟不同云存储质量）；命中本地缓存的读根本不走这条路径。于是命中/未命中的延迟差真实可控，限速参数成为实验自变量。

10.3 对照组矩阵（五组，同一负载轨迹）

#	条件	说明
1	直读 MinIO	零缓存基线
2	JuiceFS CE 冷读	首次访问，本地缓存空
3	JuiceFS CE warmup 热读	预热完成后读
4	Fluid + JuiceFSRuntime + DataLoad + 亲和调度	最强工业基线，按官方姿势操作并记录全部 yaml
5	TrainCache	required 门控 / preferred 门控 / 超时降级 各策略分别跑

五组共用固定随机种子的请求轨迹文件；同机、同冷态规程。**跑输第 4 组也是合格报告**——学习项目的原罪是不敢上秤，不是输。

10.4 硬纪律

- 1. 每组 **≥7 次取中位数，报告 bootstrap 95% CI**；3 次取中位数不进报告；
- 2. 每轮之间清页缓存（sync；echo 3 > drop_caches）+ 重启相关进程；MinIO 数据目录与缓存目录尽量分盘（外接 SSD），分不开则在报告里把共享盘列为已知混杂因素；
- 3. 完全锁定：镜像 digest、JuiceFS/Fluid 版本、tc 参数、负载配置、随机种子全部入库，`make bench COND=xxx` 一键复现；
- 4. 先写预注册假设（§ 6.2），禁止看到结果后调参”做出”目标数字；目标百分比（如 ↓60%）从文档正文中删除，只保留为假设中的预期方向。

11. 项目优点（面试可讲）

- 1. 完整 Operator 工程：CRD（etcd 尺寸约束下的 shard 状态设计）、幂等 reconcile、condition/finalizer；

- 2. 调度器框架全链路：SchedulingGate + Score（无网络调用的本地缓存范式）+ 块级子集就绪度 + 超时降级与死锁防护；
- 3. “预热-调度闭环”为现勘取证的工业空白，且配有盈亏平衡曲面——含阴性区间的诚实量化；
- 4. 双层标识模型（路径索引 + CAS 存储层）带来的内置校验与安全去重，以及”纠正自己术语错误”的真实设计过程；
- 5. 缓存准入在**对的场景**下对照（欠配+倾斜+多租户），并能讲清何时无效；
- 6. I/O 与分布式硬货：单飞行合并 + AIMD 背压、P2C 带宽预算、CAS 下的 unlink/rename 内核语义、混沌四件套；
- 7. 与真实 Fluid/JuiceFS 五组对照，改进向 Fluid 回馈——第三方背书是简历最高信用。

12. 诚实清单（不足主动说）

- 3 节点 kind：机制验证与相对趋势，不做千卡规模外推（用瓶颈模型讨论外推并标注假设）；
- 单物理盘的冷热差由 tc 受控限速构造，绝对带宽数字以一次两盘云主机校准替换；
- preheat 模型命中率为旁路口径（inotify + 消费者打点），非读路径精确记账；
- 核心 1 的”空白”判断以阶段 0 现勘为准，闸门反转则叙事收缩，不硬撞；
- 无真 RDMA/GDS 性能数据：Soft-RoCE 只验功能，eRDMA 仅一次性按小时校准；
- 元数据依赖 etcd 不自研 Raft：刻意取舍，由 S1 补能力底座；
- FUSE、自研失效协议明确不做（工业方案成熟/版本前缀已消解问题），不虚报。

13. 砍项清单（v1 → v3，说清为什么）

v1 内容	处置	理由
增量 1 gate + 增量 2 状态机	合并为核心 1 闭环，子集 readyRatio + 盈亏平衡实验	两件事本就是一个控制回路；旧指标只报赢家
“内容寻址分块”措辞	改为路径索引 + CAS 存储层	术语硬伤；顺带白赚安全去重
百万小文件为唯一/主负载	降为 A 轨；新增 B 真实分片主臂并前移步骤 5	2026 训练主流是百 MB 分片流式读
W-TinyLFU 在顺序 trace 上对比	改为欠配+倾斜+多 Job 场景，接受阴性结论	顺序场景三策略必然趋同
增量 4”权重先起先恢复”	修正：训练恢复须含 optimizer state	否则改变收敛轨迹，语义错误
增量 5 普通 P2P	重定位：CE 基线上复现 EE 缓存组语义 + Transport 抽象	对标明确，RDMA 有挂载点
增量 7 自研版本失效协议	砍，版本前缀目录 + 一页 ADR	人为造深度
FUSE	降为可选 stretch	不重造 JuiceFS/Fluid 数据面
RDMA/GDS	调研/实验臂（Soft-RoCE + 按小时租 eRDMA）	无硬件不进主路径

v1 内容	处置	理由
步骤 13 租 GPU	改为两盘 CPU 云主机存储真值校准	P1 全项目不需要 GPU，省钱

14. 实施步骤（14 步，闸门驱动）

步骤	内容	验收 / 注意点
0 现勘	§ 4: 4.0 可行性探针（2-3 天，最先做） + kind + MinIO + JuiceFS CE + 开源 Fluid 部署，DataLoad/亲和 demo，取证注入与时序，现勘报告	双闸门： 探针 A/B 绿灯（装置可信）+ 缺口被证据确认，才进入核心 1 闭环叙事；任一不满足先修装置/收缩叙事，不带病开工；与 P2 现勘同期推进
1 实验装置	tc tbf/netem 限速规程 + drop_caches 规程 + 三轨数据集生成器	make bench-up；限速档位可扫点，冷态可重复
2 缓存核心库	manifest 路径索引 + CAS 存储层（流式摘要/refcount）+ 限并发抓取	单测：并发写同块、tmp-rename 原子性、同内容去重、校验失败重拉
3 Agent 服务化	gRPC（预热/状态）+ /metrics + 启动重扫描 + inotify	指标带 dataset 标签防基数爆炸；重启恢复有测试
4 数据面	DaemonSet + hostPath 只读约定 + 预热 CLI	手动预热后普通 Pod 全量可读
5 Controller + CRD + 真实 dataloader	reconcile 分发回填；webdataset/MDS + PyTorch DataLoader 跑通 B 轨	端到端 MVP；informer 重入幂等； 真实负载在此接入，不后置
6 基准框架与 Fluid 上秤	五组对照自动化（至少先通 1/2/3/5 组），≥7 次 + CI，一键出图	第一个简历成果： 含对手的对照报告
7 核心 1 闭环	gate + Score + 子集 readyRatio + 候选放置 + 超时降级	旗舰图： 盈亏平衡曲面 + fanout Pareto；plugin 无网络调用
8 核心 2 准入预取	C 轨多 Job trace；LRU/LFU/W-TinyLFU + 自适应预取	倾斜场景差异曲线 + 顺序场景阴性结论 + 参数敏感性
9 S3 网关	singleflight + io.Pipe 扇出 + AIMD	大对象扇出内存不涨（压测证明）
10 Checkpoint sidecar	multipart + manifest 发布点 + 修正后的恢复语义	训练恢复必须状态完整；恢复时长拆解
11 P2P 缓存组	哈希环 + peer 回源 + 带宽预算 + Transport 接口	源站出口字节下降；origin 保底生效
12 混沌套件	§ 8.4 四件套全绿	恢复时间指标齐备
13 真值校准	两独立数据盘 CPU 云主机 + tc 校准，五组对照全量复跑	真存储数字替换单机数字，双版保留说明差异
14 上游回馈	以 § 6 数据向 Fluid 提 issue → 讨论 → PR（调度/DataLoad 方向）	≥1 个带数据的 issue；代码 PR 按维护者认可推进

RDMA 实验臂 (§ 9.1) 挂在步骤 11 之后按兴趣与预算进行，不占关键路径；io_uring (S2) 在步骤 8 前后以 A 轨数据独立推进。

15. 排期与死线（倒推，与 P2 共用日历）

- **2027.2 实习投递包最低构成**：步骤 0–7 + S1 (mini-Raft) + ≥ 1 个带测量数据的 Fluid issue。即：基座 + 核心 1（五组对照至少三组 + 盈亏平衡旗舰图）；
- **2027.8 秋招包**：步骤 0–11 + 13 + 14 (issue 有维护者正式回应或 PR 进展)；
- 砍项顺序（时间不够时从后往前砍）：RDMA 臂 → FUSE → 混沌的位翻转用例 → P2P 高级部分（多源调度）→ W-TinyLFU 的 Count-Min 完整版（SlidingWindow LRU 足够进简历）；**核心 1、B 轨真实负载、Fluid 对照不砍**；
- 阶段 0 与 P2 现勘共用”使用阶段”，立即启动。

16. 预期成果

交付物：TrainCache 仓库（代码 + README + ADR + 复现命令）、《P1 对手现勘报告》、五组对照实验报告（含盈亏平衡曲面与阴性区间）、Grafana 看板、博客 2 篇（调度-预热闭环实测 / W-TinyLFU 在什么场景下才有用）、Fluid issue/PR 链接。

简历条目草稿 (V1 主版)：

设计并实现 K8s 原生训练数据预热-调度闭环：CRD/Operator + SchedulingGate 门控 + 基于 Job 数据子集块级就绪度的调度打分与超时降级；路径索引 + CAS 内容摘要存储层（内置校验与安全去重）；3 节点 kind 下与真实 Fluid/JuiceFS 五组对照，以端到端 makespan 给出预热门控相对自由调度的盈亏边界（源站带宽 $\times$ 缓存欠配 $\times$ fanout），重复 epoch/多 Job 共享场景 makespan $\downarrow$ X%，并量化其在单次顺序扫描下的阴性区间；欠配缓存多租户场景实测 W-TinyLFU/LRU 准入差异；相关测量以 issue/PR 回馈 Fluid 社区。